Unit 5 – Stage Decision Problems and the Linear Quadratic Regulator

4SC000 Optimal Control and Reinforcement Learning

Duarte Antunes

Slides · Exercises · Jupyter notebooks

Stage decision problems

This unit opens Part II of the course. Units 1-4 dealt with *discrete optimization problems*: the state and input spaces were finite or countable, and a problem instance could be drawn as a transition diagram and solved with the dynamic programming algorithm or, for shortest-path problems, with Dijkstra's algorithm or A*. Units 5-6 study *stage decision problems*, where the state and input spaces can be arbitrary – typically $\mathbb{R}^n$ and $\mathbb{R}^m$ – so the underlying system is a genuine dynamic system rather than a graph. The table below summarizes how the concepts developed for discrete optimization problems carry over.

	Discrete optimization problems	**Stage decision problems**
Formulation	Transition diagram	Dynamic system & additive cost function
DP algorithm & Stochastic DP	Graphical DP algorithm & DP equation	DP equation
Alternative algorithms	Dijkstra's algorithm	Static optimization
Partial information	Bayes filter	Kalman filter
Application focus	Operational research & Computer science	Digital control

This unit covers the first row and the DP-equation entry of the second row, specialized to the most important tractable case: linear dynamics and a quadratic cost, which leads to the *linear quadratic regulator* (LQR), one of the cornerstone results of optimal control. Unit 6 covers static optimization as an alternative to DP, and the Kalman filter, the counterpart of the Bayes' filter for systems with continuous state.

Dynamic model and cost function

A stage decision problem is described by a dynamic model

$$x_{k+1} = f_k(x_k, u_k), \qquad k \in \{0, \dots, h-1\},$$

and an additive cost function

$$\sum_{k=0}^{h-1} g_k(x_k, u_k) + g_h(x_h).$$

The state $x_k \in X_k$ and input $u_k \in U_k(x_k)$ live in arbitrary spaces. Whenever these spaces are discrete, we recover the discrete optimization problems of Units 1-4; here we are typically interested in the case $x_k \in \mathbb{R}^n$ and $u_k \in \mathbb{R}^m$ for every stage $k \in \{0, \dots, h-1\}$. As before, the two problems of interest are computing an optimal path for a given initial condition, and computing an optimal policy that works for every initial condition.

Optimal paths and policies

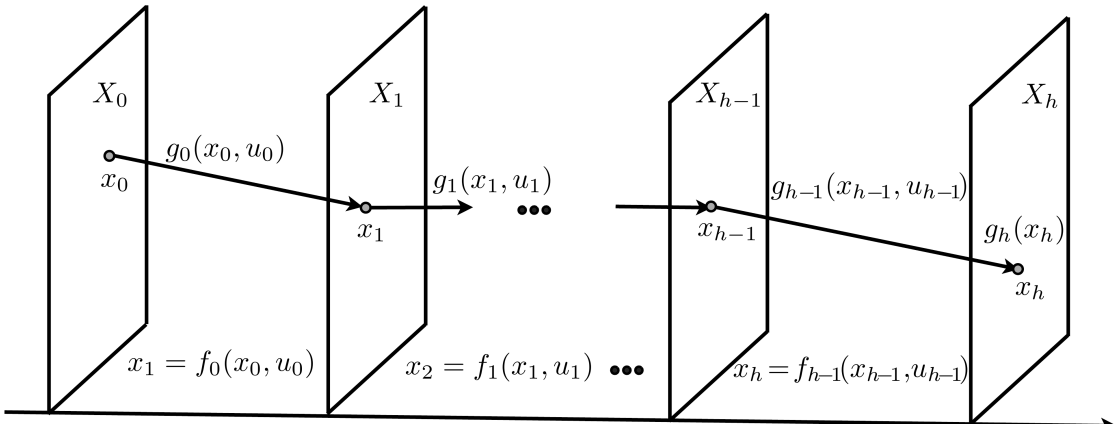


Fig. 1: The state spaces $X_0, \dots, X_h$ at each stage, with a path $x_0, \dots, x_h$ connecting them. Each transition $x_k \to x_{k+1}$ is generated by a decision u_k, incurs stage cost $g_k(x_k, u_k)$, and follows the dynamics $x_{k+1} = f_k(x_k, u_k)$; the terminal state x_h incurs the terminal cost $g_h(x_h)$.

Given an initial condition x_0, a *path* is a set of decisions $\{u_0, u_1, \dots, u_{h-1}\}$ such that $u_k \in U_k(x_k)$ for every k and the resulting state sequence $\{(x_0, u_0), \dots, (x_{h-1}, u_{h-1})\}$ satisfies the equations of the dynamic model. A path is *optimal* if there does not exist another path, with the same initial condition, that has a strictly smaller cost.

A *policy* is a set of h functions $\pi = \{\mu_0, \dots, \mu_{h-1}\}$, $\mu_k : X_k \to U_k$, one for each stage: rather than fixing a sequence of decisions, a policy tells the decision-maker what to do from any state it might find itself in at any stage. A policy is *optimal* if, for every state x_ℓ at every stage $\ell \in \{0, \dots, h-1\}$, the action $\mu_\ell(x_\ell)$ is the first action of an optimal path for the *tail subproblem* that considers only stages $\{\ell, \ell+1, \dots, h\}$, with initial condition x_ℓ and cost

$$\sum_{k=\ell}^{h-1} g_k(x_k, u_k) + g_h(x_h).$$

This is exactly the same optimal-path/optimal-policy distinction used throughout Units 1-2, now stated for arbitrary state and input spaces.

The dynamic programming algorithm

The dynamic programming algorithm computes an optimal policy exactly as it did for discrete optimization problems. Start with $J_h(x_h) = g_h(x_h)$ for every $x_h \in X_h$, and for each decision stage, starting at the last and moving backwards, $k \in \{h-1, h-2, \dots, 0\}$, compute μ_k and J_k from the *DP equation*

$$J_k(x_k) = \min_{u_k \in U_k(x_k)} g_k(x_k, u_k) + J_{k+1}(f_k(x_k, u_k)),$$

and

$$\mu_k(x_k) = u_k,$$

where u_k is a minimizer in the DP equation, i.e.,

$$J_k(x_k) = g_k(x_k, \mu_k(x_k)) + J_{k+1}(f_k(x_k, \mu_k(x_k))).$$

Then $\{\mu_0, \dots, \mu_{h-1}\}$ is an optimal policy.

Theorem. The policy obtained with the DP algorithm is an optimal policy.

Proof of optimality of dynamic programming

The proof is by induction on k, and shows that $\pi^k := \{\mu_k, \dots, \mu_{h-1}\}$, obtained from the DP algorithm, is an optimal policy for the subproblem running from stage k to stage $h-1$, and that $J_k(x_k)$ is the cost of the optimal path starting at x_k.
Step I ($k = h-1$). By construction $\pi^{h-1} = \{\mu_{h-1}\}$ is an optimal policy, since it is by definition the first (and only) decision of the optimal path from stage $h-1$ to stage h:

$$\min_{u_{h-1} \in U_{h-1}(x_{h-1})} g_{h-1}(x_{h-1}, u_{h-1}) + J_h(f_{h-1}(x_{h-1}, u_{h-1})) = g_{h-1}(x_{h-1}, \mu_{h-1}(x_{h-1})) + J_h(f_{h-1}(x_{h-1}, \mu_{h-1}(x_{h-1}))).$$

It is also clear that

$$J_{h-1}(x_{h-1}) = \min_{u_{h-1} \in U_{h-1}(x_{h-1})} g_{h-1}(x_{h-1}, u_{h-1}) + J_h(f_{h-1}(x_{h-1}, u_{h-1}))$$

is the optimal cost of the subproblem with initial condition x_{h-1} at stage $h-1$.
Step II (inductive step). Assume that $\pi^{k+1} := \{\mu_{k+1}, \dots, \mu_{h-1}\}$ is an optimal policy and that $J_{k+1}(x_{k+1})$ is the cost of the optimal path starting at x_{k+1}. We show, by contradiction, that $\pi^k := \{\mu_k, \dots, \mu_{h-1}\}$ is an optimal policy and that $J_k(x_k)$ is the cost of an optimal path starting at x_k.
Suppose π^k is not optimal. Then there is a state x_k for which $\mu_k(x_k)$ is *not* the first action of an optimal path from stage k to stage h. Call that optimal path

$$\tilde{\gamma} = \{(\tilde{x}_k, \tilde{u}_k), (\tilde{x}_{k+1}, \tilde{u}_{k+1}), \dots, \tilde{x}_h)\}, \qquad \tilde{x}_k = x_k, \quad \tilde{u}_k \neq \mu_k(x_k).$$

Because π^{k+1} is, by the induction hypothesis, an optimal policy for the tail subproblem starting at stage $k+1$, the remainder of $\tilde{\gamma}$ must in fact agree with it:

$$\tilde{u}_{\ell+1} = \mu_{\ell+1}(\tilde{x}_{\ell+1}) \quad \text{for every } \ell \in \{k+1, \dots, h-2\}.$$

The cost of $\tilde{\gamma}$ is therefore

$$\bar{J} = \sum_{\ell=k}^{h-1} g_\ell(\tilde{x}_\ell, \tilde{u}_\ell) + g_h(\tilde{x}_h) = g_k(\tilde{x}_k, \tilde{u}_k) + \sum_{\ell=k+1}^{h-1} g_\ell(\tilde{x}_\ell, \mu_\ell(\tilde{x}_\ell)) + g_h(\tilde{x}_h) = g_k(\tilde{x}_k, \tilde{u}_k) + J_{k+1}(f(\tilde{x}_k, \tilde{u}_k)).$$

But by definition $\mu_k(x_k)$ minimizes exactly this quantity over u_k, so

$$J_k(x_k) = \min_{u_k} g_k(\tilde{x}_k, u_k) + J_{k+1}(f(\tilde{x}_k, u_k)) = g_k(\tilde{x}_k, \mu_k(x_k)) + J_{k+1}(f(\tilde{x}_k, \mu_k(x_k))) \leq g_k(\tilde{x}_k, \tilde{u}_k) + J_{k+1}(f(\tilde{x}_k, \tilde{u}_k)) = \bar{J},$$

which contradicts the assumption that $\tilde{\gamma}$, with first action $\tilde{u}_k \neq \mu_k(x_k)$, is strictly better than the path generated by μ_k. Hence π^k is optimal and $J_k(x_k)$ is the optimal cost, completing the induction.

A worked example: quadratic vs. non-quadratic terminal cost

To see the DP algorithm at work on a stage decision problem, and to motivate why we will restrict attention to linear dynamics and quadratic costs, consider the simple integrator

$$x_{k+1} = x_k + u_k, \qquad k \in \{0, 1\},$$

with cost

$$\sum_{k=0}^{1} x_k^2 + u_k^2 + g_2(x_2),$$

and compare a quadratic terminal cost $g_2(x_2) = x_2^2$ with a non-quadratic one, $g_2(x_2) = e^{x_2}$.

Quadratic terminal cost. At $k = 1$, using $x_2 = x_1 + u_1$,

$$J_1(x_1) = \min_{u_1} g_1(x_1, u_1) + g_2(x_2) = \min_{u_1} x_1^2 + u_1^2 + x_2^2 = \min_{u_1} \underbrace{2(x_1^2 + u_1^2 + x_1 u_1)}_{Q_{x_1}(u_1),\ \text{quadratic function of } u_1}.$$

For a fixed x_1, $Q_{x_1}(u_1)$ is a strictly convex parabola in u_1, so its minimum can be found by differentiating and equating to zero, as illustrated below for a particular value of x_1.

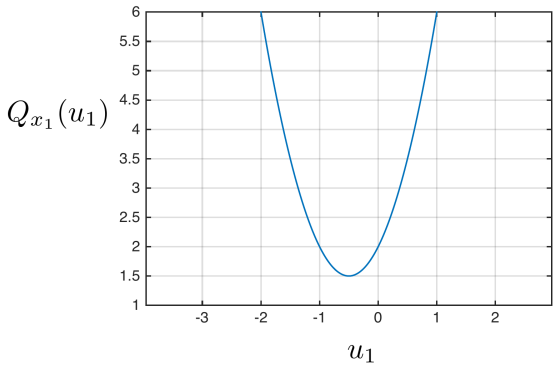


Fig. 2: The cost-to-go $Q_{x_1}(u_1)$ as a function of u_1, for a fixed value of x_1: a strictly convex parabola with a unique minimizer.

$$\frac{d}{du_1} Q_{x_1}(u_1) = 0 \iff 2(2u_1 + x_1) = 0 \iff u_1 = -\frac{1}{2} x_1.$$

Substituting back into $Q_{x_1}(u_1)$ gives the cost-to-go $J_1(x_1) = \frac{3}{2} x_1^2$, which is again quadratic in x_1. This is the key feature that makes the recursion tractable: at $k = 0$, using $x_1 = x_0 + u_0$,

$$J_0(x_0) = \min_{u_0} g_0(x_0, u_0) + J_1(x_1) = \min_{u_0} x_0^2 + u_0^2 + \frac{3}{2} x_1^2 = \min_{u_0} \frac{5}{2} x_0^2 + 3 u_0 x_0 + \frac{5}{2} u_0^2,$$

and differentiating and equating to zero gives a second function of the optimal policy,

$$u_0 = -\frac{3}{5} x_0, \qquad J_0(x_0) = \frac{8}{5} x_0^2.$$

The optimal policy is therefore $u_0 = -\frac{3}{5}x_0$, $u_1 = -\frac{1}{2}x_1$. Starting from $x_0 = 1$ and iterating $x_{k+1} = x_k + u_k$ gives the optimal path

$$x_0 = 1, \quad u_0 = -\frac{3}{5}, \qquad x_1 = \frac{2}{5}, \quad u_1 = -\frac{1}{5}, \qquad x_2 = \frac{1}{5},$$

with optimal cost $J_0(1) = \frac{8}{5}$.

Non-quadratic terminal cost. Repeat the same first step with $g_2(x_2) = e^{x_2}$:

$$J_1(x_1) = \min_{u_1} g_1(x_1, u_1) + g_2(x_2) = \min_{u_1} x_1^2 + u_1^2 + e^{x_2}.$$

Differentiating and equating to zero now gives

$$2u_1 + e^{x_1+u_1} = 0,$$

an equation that implicitly determines u_1 in terms of x_1, but has no closed-form solution. Consequently, we cannot write down $u_1 = \mu_1(x_1)$ in closed form, and so we cannot move on to the second step of the DP recursion the way we could in the quadratic case.

This little example illustrates a general phenomenon. When the dynamic model is linear and the cost is quadratic, the cost-to-go remains quadratic at every stage, so the DP recursion reduces to simple algebra on the coefficients of that quadratic form, and the optimal policy can be obtained explicitly, as we show next. When the dynamic model is non-linear and/or the cost is non-quadratic, applying DP explicitly is generally very hard; one has to resort to approximation techniques such as discretizing the state and input spaces (turning the problem back into a discrete optimization problem), or other approximation techniques that will be discussed in later lectures.

Quick exercise: the same recursion on a different problem

Repeat the quadratic calculation above for the stage decision problem

$$x_{k+1} = 2x_k - u_k, \qquad k \in \{0, 1\},$$

with cost

$$\sum_{k=0}^{1} \left(3x_k^2 + \tfrac{1}{2}u_k^2\right) + x_2^2.$$

Compute the optimal policy $u_0 = \mu_0(x_0)$, $u_1 = \mu_1(x_1)$ and the costs-to-go J_0, J_1, and then the optimal path and its cost for the initial condition $x_0 = -2$.

Solution

Stage $k = 1$. Substituting $x_2 = 2x_1 - u_1$,

$$J_1(x_1) = \min_{u_1} \; 3x_1^2 + \tfrac{1}{2}u_1^2 + (2x_1 - u_1)^2 = \min_{u_1} \; \underbrace{7x_1^2 - 4x_1u_1 + \tfrac{3}{2}u_1^2}_{\text{quadratic in } u_1},$$

which is again a strictly convex parabola in u_1, so differentiating and equating to zero gives

$$-4x_1 + 3u_1 = 0 \iff u_1 = \tfrac{4}{3}x_1.$$

Substituting back,

$$J_1(x_1) = \left(7 - \tfrac{16}{3} + \tfrac{8}{3}\right)x_1^2 = \tfrac{13}{3}x_1^2,$$

quadratic in x_1, exactly as in the worked example.
Stage $k = 0$. Substituting $x_1 = 2x_0 - u_0$,

$$J_0(x_0) = \min_{u_0}\ 3x_0^2 + \tfrac{1}{2}u_0^2 + \tfrac{13}{3}(2x_0 - u_0)^2 = \min_{u_0}\ \tfrac{61}{3}x_0^2 - \tfrac{52}{3}x_0u_0 + \tfrac{29}{6}u_0^2,$$

and differentiating and equating to zero,

$$-\tfrac{52}{3}x_0 + \tfrac{29}{3}u_0 = 0 \iff u_0 = \tfrac{52}{29}x_0, \qquad J_0(x_0) = \tfrac{139}{29}x_0^2.$$

So the optimal policy is

$$\mu_0(x_0) = \tfrac{52}{29}x_0, \qquad \mu_1(x_1) = \tfrac{4}{3}x_1.$$

Note that the gains are *positive* here: the open-loop dynamics $x_{k+1} = 2x_k$ are unstable and the input enters with a minus sign, so the controller must push with the same sign as the state in order to oppose the growth.
Optimal path for $x_0 = -2$. Iterating $x_{k+1} = 2x_k - u_k$,

$$x_0 = -2, \quad u_0 = -\tfrac{104}{29} \approx -3.5862, \qquad x_1 = -\tfrac{12}{29} \approx -0.4138, \quad u_1 = -\tfrac{16}{29} \approx -0.5517, \qquad x_2 = -\tfrac{8}{29} \approx -0.2759,$$

with optimal cost

$$J_0(-2) = \tfrac{139}{29} \cdot 4 = \tfrac{556}{29} \approx 19.1724.$$

As a check, evaluating the cost directly along this path gives $3(4) + \tfrac{1}{2}\left(\tfrac{104}{29}\right)^2 + 3\left(\tfrac{12}{29}\right)^2 + \tfrac{1}{2}\left(\tfrac{16}{29}\right)^2 + \left(\tfrac{8}{29}\right)^2 = \tfrac{556}{29}$, the same value.

The linear quadratic regulator

Motivated by the previous example, we now specialize to linear dynamics and a quadratic cost. Given a dynamic model

$$x_{k+1} = A_k x_k + B_k u_k, \qquad k \in \{0, \dots, h-1\},$$

and a cost function

$$\sum_{k=0}^{h-1} \begin{bmatrix} x_k^\top & u_k^\top \end{bmatrix} \begin{bmatrix} Q_k & S_k \\ S_k^\top & R_k \end{bmatrix} \begin{bmatrix} x_k \\ u_k \end{bmatrix} + x_h^\top Q_h x_h,$$

the goal is to find an optimal policy $u_k = \mu_k(x_k)$, $k \in \{0, \dots, h-1\}$. This is the finite-horizon, discrete-time linear quadratic optimal control problem. When $h \to \infty$ and the matrices in the model and cost are time-invariant, the resulting solution is called the *linear quadratic regulator* (LQR) – one of the celebrated results of control theory, and one of the main achievements of optimal control.

Throughout, Q_k and R_k are assumed symmetric, with $\begin{bmatrix} Q_k & S_k \\ S_k^\top & R_k \end{bmatrix} \geq 0$, $Q_h \geq 0$ and $R_k > 0$. The model and cost are often time-invariant, i.e., $x_{k+1} = Ax_k + Bu_k$ and

$$\sum_{k=0}^{h-1} \begin{bmatrix} x_k^\top & u_k^\top \end{bmatrix} \begin{bmatrix} Q & S \\ S^\top & R \end{bmatrix} \begin{bmatrix} x_k \\ u_k \end{bmatrix} + x_h^\top Q_h x_h.$$

The cost function can arise from discretizing a continuous-time problem, but in general it is directly specified in discrete time and used as a tuning knob to obtain desired closed-loop specifications, such as overshoot or settling time. We focus throughout on the *stabilization* problem, i.e., driving the state to zero; for the reference-tracking problem, see the accompanying notes, exercises and Jupyter notebooks.

Deriving the Riccati recursion

We now apply the DP algorithm to this problem. At the last stage, $x_h = A_{h-1}x_{h-1} + B_{h-1}u_{h-1}$, so

$$J_{h-1}(x_{h-1}) = \min_{u_{h-1}} \begin{bmatrix} x_{h-1}^\top & u_{h-1}^\top \end{bmatrix} \begin{bmatrix} Q_{h-1} & S_{h-1} \\ S_{h-1}^\top & R_{h-1} \end{bmatrix} \begin{bmatrix} x_{h-1} \\ u_{h-1} \end{bmatrix} + \underbrace{J_h(x_h)}_{x_h^\top Q_h x_h}.$$

Substituting $x_h = A_{h-1}x_{h-1} + B_{h-1}u_{h-1}$ into the terminal cost and expanding, the expression above becomes

$$J_{h-1}(x_{h-1}) = \min_{u_{h-1}} x_{h-1}^\top (A_{h-1}^\top Q_h A_{h-1} + Q_{h-1}) x_{h-1} + 2u_{h-1}^\top (S_{h-1}^\top + B_{h-1}^\top Q_h A_{h-1}) x_{h-1} + \underbrace{u_{h-1}^\top (B_{h-1}^\top Q_h B_{h-1} + R_{h-1}) u_{h-1}}_{\text{quadratic function of } u_{h-1}}.$$

Minimizing over u_{h-1} therefore amounts to minimizing a general quadratic function of a vector variable, so it is worth solving that problem once and for all.

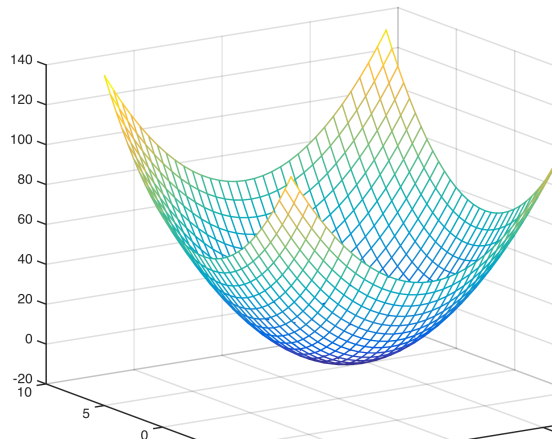


Fig. 3: A quadratic function $J(u) = u^\top Xu + 2u^\top y + z$ of two variables, with $X > 0$: the surface is a bowl with a unique minimizer, found by setting the gradient to zero.

For $J(u) = u^\top Xu + 2u^\top y + z$ with $X > 0$,

$$\nabla J(u) = 0 \iff 2Xu + 2y = 0 \iff u = -X^{-1}y,$$

and this is the unique minimizer since $X > 0$. The minimum value is

$$J(-X^{-1}y) = y^\top X^{-1} X X^{-1} y - 2y^\top X^{-1} y + z = z - y^\top X^{-1} y.$$

Applying this with

$$X = B_{h-1}^\top Q_h B_{h-1} + R_{h-1}, \qquad y = S_{h-1}^\top + B_{h-1}^\top Q_h A_{h-1}, \qquad z = A_{h-1}^\top Q_h A_{h-1} + Q_{h-1}$$

(the last two understood as quadratic forms in x_{h-1}), we obtain the policy

$$u_{h-1} = -X^{-1}y = -(B_{h-1}^\top Q_h B_{h-1} + R_{h-1})^{-1}(S_{h-1}^\top + B_{h-1}^\top Q_h A_{h-1})x_{h-1}$$

and the cost-to-go

$$J_{h-1}(x_{h-1}) = x_{h-1}^\top \Big[A_{h-1}^\top Q_h A_{h-1} + Q_{h-1} - (S_{h-1} + A_{h-1}^\top Q_h B_{h-1})(B_{h-1}^\top Q_h B_{h-1} + R_{h-1})^{-1}(S_{h-1}^\top + B_{h-1}^\top Q_h A_{h-1})\Big] x_{h-1}.$$

Crucially, J_{h-1} is again a quadratic form, $J_{h-1}(x_{h-1}) = x_{h-1}^\top P_{h-1} x_{h-1}$, with P_{h-1} the bracketed matrix above. This means the next step of the DP recursion, at $k = h-2$, has exactly the same structure as the first one, with Q_h replaced by P_{h-1}:

$$J_{h-2}(x_{h-2}) = \min_{u_{h-2}} \begin{bmatrix} x_{h-2}^\top & u_{h-2}^\top \end{bmatrix} \begin{bmatrix} Q_{h-2} & S_{h-2} \\ S_{h-2}^\top & R_{h-2} \end{bmatrix} \begin{bmatrix} x_{h-2} \\ u_{h-2} \end{bmatrix} + \underbrace{J_{h-1}(x_{h-1})}_{x_{h-1}^\top P_{h-1} x_{h-1}},$$

so the same reasoning gives $u_{h-2} = K_{h-2}x_{h-2}$, $J_{h-2}(x_{h-2}) = x_{h-2}^\top P_{h-2} x_{h-2}$, with

$$K_{h-2} = -(B_{h-2}^\top P_{h-1} B_{h-2} + R_{h-2})^{-1}(S_{h-2}^\top + B_{h-2}^\top P_{h-1} A_{h-2}),$$

$$P_{h-2} = A_{h-2}^\top P_{h-1} A_{h-2} + Q_{h-2} - (S_{h-2} + A_{h-2}^\top P_{h-1} B_{h-2})(B_{h-2}^\top P_{h-1} B_{h-2} + R_{h-2})^{-1}(S_{h-2}^\top + B_{h-2}^\top P_{h-1} A_{h-2}).$$

Since the cost-to-go stays quadratic at every stage, the same computation repeats all the way back to $k = 0$: at a generic stage k, with $x_{k+1} = A_k x_k + B_k u_k$,

$$u_k = K_k x_k, \qquad J_k(x_k) = x_k^\top P_k x_k,$$

$$K_k = -(B_k^\top P_{k+1} B_k + R_k)^{-1}(S_k^\top + B_k^\top P_{k+1} A_k),$$

$$P_k = A_k^\top P_{k+1} A_k + Q_k - (S_k + A_k^\top P_{k+1} B_k)(B_k^\top P_{k+1} B_k + R_k)^{-1}(S_k^\top + B_k^\top P_{k+1} A_k) \qquad \text{(Riccati equation)}.$$

To compute the optimal policy, simply iterate these two equations backwards for $k \in \{h-1, \dots, 1, 0\}$, starting from $P_h = Q_h$. The optimal cost for a given initial condition is $J_0(x_0) = x_0^\top P_0 x_0$.

Example: the double integrator

Consider a mass sliding on a frictionless surface, actuated by a horizontal force F, with position y measured from a reference point:

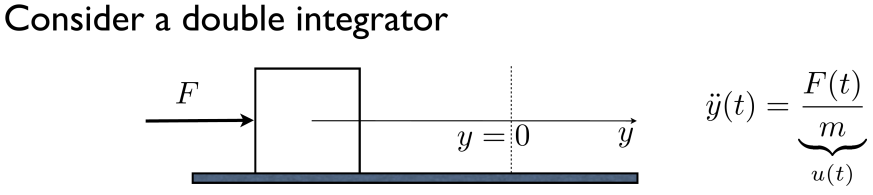


Fig. 4: A double integrator: a mass m on a frictionless surface, actuated by a force $F(t)$, with $\ddot{y}(t) = F(t)/m = u(t)$.

Taking $u(t) = F(t)/m$ as the control input, the continuous-time model in state-space form, with $x(t) = [y(t) \;\; v(t)]^\top$, is

$$\begin{bmatrix}\dot{y}(t)\\ \dot{v}(t)\end{bmatrix} = \begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}\begin{bmatrix}y(t)\\ v(t)\end{bmatrix} + \begin{bmatrix}0\\ 1\end{bmatrix} u(t).$$

Discretizing with sampling period τ gives the discrete-time model

$$x_{k+1} = e^{\begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}\tau} x_k + \int_0^\tau e^{\begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix} r} dr \begin{bmatrix}0\\ 1\end{bmatrix} u_k = \begin{bmatrix}1 & \tau\\ 0 & 1\end{bmatrix} x_k + \begin{bmatrix}\frac{\tau^2}{2}\\ \tau\end{bmatrix} u_k.$$

Take $\tau = 0.2$. The qualitative goal is to drive the mass to position zero quickly, but with reasonable actuation. As a first attempt, use the cost function

$$\sum_{k=0}^{h-1} x_k^\top Q x_k + u_k^\top R u_k + x_h^\top Q_h x_h, \qquad Q = \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}, \quad R = 1, \quad h = 5, \quad Q_h = \begin{bmatrix}10 & 0\\ 0 & 10\end{bmatrix},$$

and tune the weights afterwards to improve the closed-loop behavior.

Iterating the Riccati recursion backwards from $P_5 = Q_5 = \begin{bmatrix}10 & 0\\ 0 & 10\end{bmatrix}$, the first step gives

$$K_4 = -\Big(\begin{bmatrix}0.02 & 0.2\end{bmatrix} \begin{bmatrix}10 & 0\\ 0 & 10\end{bmatrix} \begin{bmatrix}0.02\\ 0.2\end{bmatrix} + 1\Big)^{-1} \begin{bmatrix}0.02 & 0.2\end{bmatrix} \begin{bmatrix}10 & 0\\ 0 & 10\end{bmatrix} \begin{bmatrix}1 & 0.2\\ 0 & 1\end{bmatrix} = \begin{bmatrix}-0.1425 & -1.4530\end{bmatrix},$$

$$P_4 = \begin{bmatrix}1 & 0.2\\ 0 & 1\end{bmatrix}^\top \begin{bmatrix}10 & 0\\ 0 & 10\end{bmatrix} \begin{bmatrix}1 & 0.2\\ 0 & 1\end{bmatrix} + \begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix} - \begin{bmatrix}1 & 0.2\\ 0 & 1\end{bmatrix}^\top \begin{bmatrix}10 & 0\\ 0 & 10\end{bmatrix} \begin{bmatrix}0.02\\ 0.2\end{bmatrix} (-K_4) = \begin{bmatrix}10.9715 & 1.7094\\ 1.7094 & 8.4359\end{bmatrix}.$$

Continuing the recursion down to $k = 0$ gives

$$P_3 = \begin{bmatrix}11.739 & 3.144\\ 3.144 & 8.0782\end{bmatrix}, \quad P_2 = \begin{bmatrix}12.188 & 4.311\\ 4.311 & 8.2725\end{bmatrix}, \quad P_1 = \begin{bmatrix}12.295 & 5.165\\ 5.165 & 8.675\end{bmatrix}, \quad P_0 = \begin{bmatrix}12.121 & 5.702\\ 5.702 & 9.085\end{bmatrix},$$

$$K_3 = [-0.414 \;\; -1.353], \quad K_2 = [-0.638 \;\; -1.368], \quad K_1 = [-0.807 \;\; -1.432], \quad K_0 = [-0.918 \;\; -1.503],$$

so the optimal policy is $u_k = K_k x_k$, $k \in \{0, 1, \dots, 4\}$. Starting from $x_0 = [1 \;\; 0]^\top$ and iterating $x_{k+1} = Ax_k + Bu_k$, $u_k = K_k x_k$, the optimal path is

$$(x_0, u_0) = ([1 \;\; 0]^\top, -0.918), \qquad (x_1, u_1) = ([0.982 \;\; -0.184]^\top, -0.529), \qquad (x_2, u_2) = ([0.934 \;\; -0.289]^\top, -0.200),$$

$$(x_3, u_3) = ([0.8724 \;\; -0.330]^\top, 0.085), \qquad (x_4, u_4) = ([0.8082 \;\; -0.313]^\top, 0.339), \qquad x_5 = [0.7525 \;\; -0.2448]^\top.$$

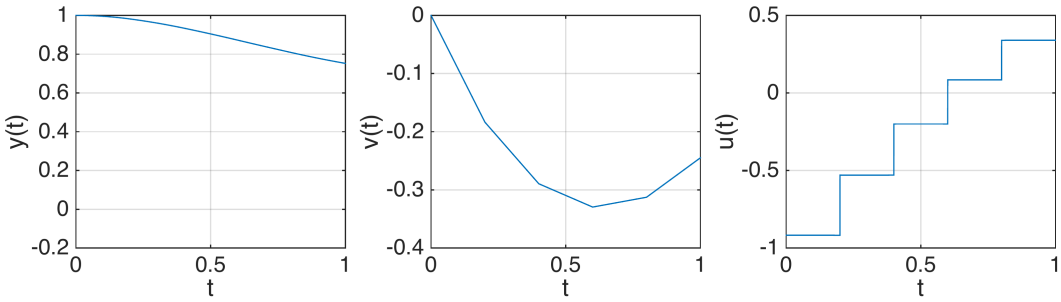
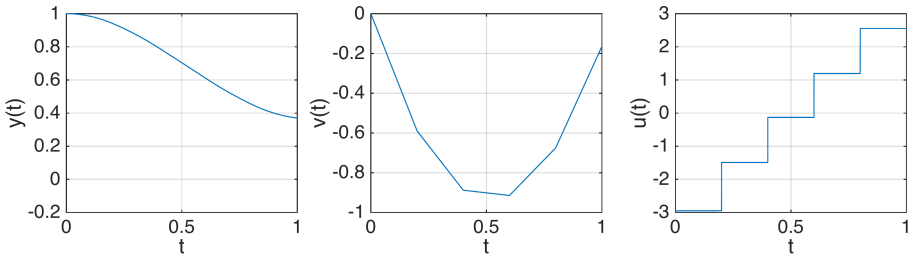


Fig. 5: Position $y(t)$, velocity $v(t)$ and control $u(t)$ for the optimal path above. With this first choice of weights the transient response is still far from the qualitative goal: the mass has only reached $y \approx 0.75$ by the end of the horizon.

The following general guidelines help tune the cost weights: increasing the terminal cost Q_h should bring the response closer to the desired final position; penalizing position error more heavily relative to velocity error has a similar effect; and decreasing the penalty on the control action R gives the controller more authority to reach the origin quickly. The next four figures illustrate each of these effects in turn, always for the same double integrator and horizon $h = 5$.

$Q_h = 100I$



$Q_h = 1000I$

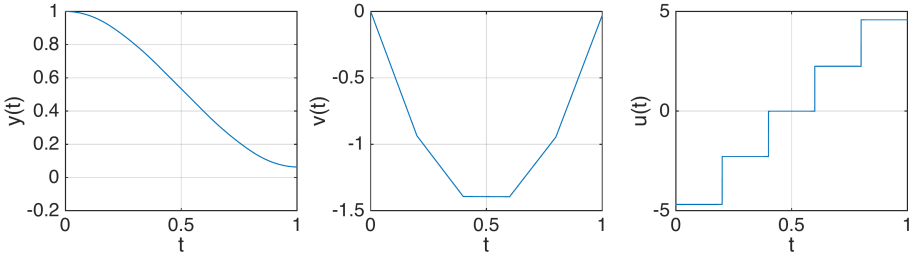


Fig. 6: Increasing the terminal cost from $Q_h = 100I$ (top) to $Q_h = 1000I$ (bottom) improves the final position error: $y(1)$ drops from about 0.37 to about 0.06, at the cost of larger control effort.

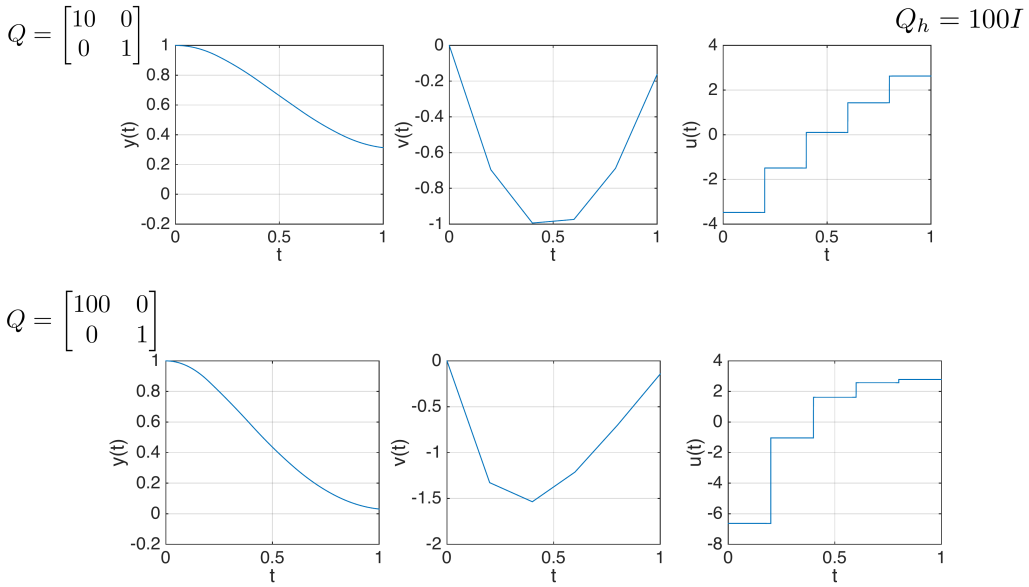


Fig. 7: Increasing the weight on position error, from $Q = \mathrm{diag}(10, 1)$ (top) to $Q = \mathrm{diag}(100, 1)$ (bottom), with $Q_h = 100I$ fixed: the position error shrinks, but the velocity error grows, since the cost no longer penalizes it as strongly in relative terms.

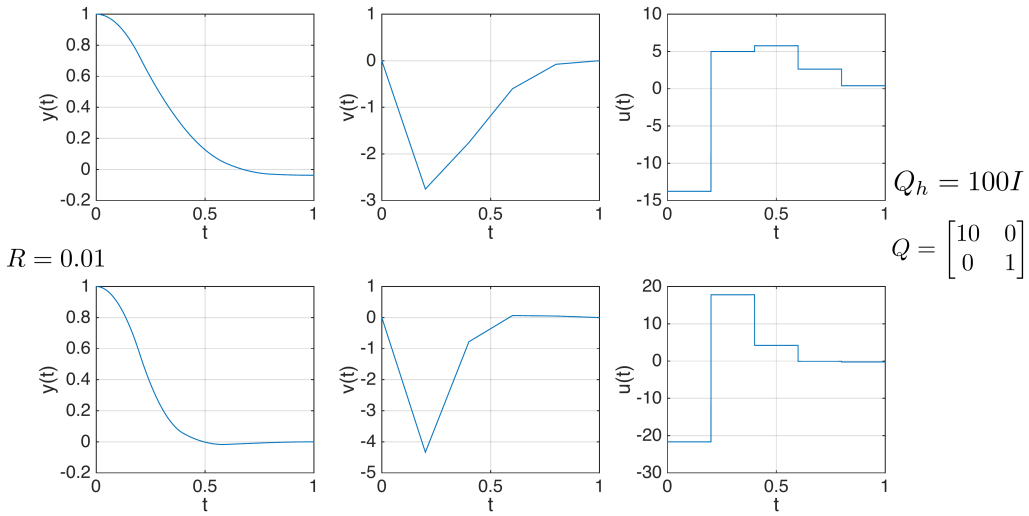


Fig. 8: Decreasing the control penalty, from $R = 1$ (top) to $R = 0.01$ (bottom), with $Q = \mathrm{diag}(10, 1)$ and $Q_h = 100I$ fixed: the response settles much faster, but at the cost of far larger actuation (peak control around -21 instead of -14).

Pushing the control penalty to its limit, as $R \to 0$ we obtain *deadbeat control*: the state is driven very close to zero in just two steps.

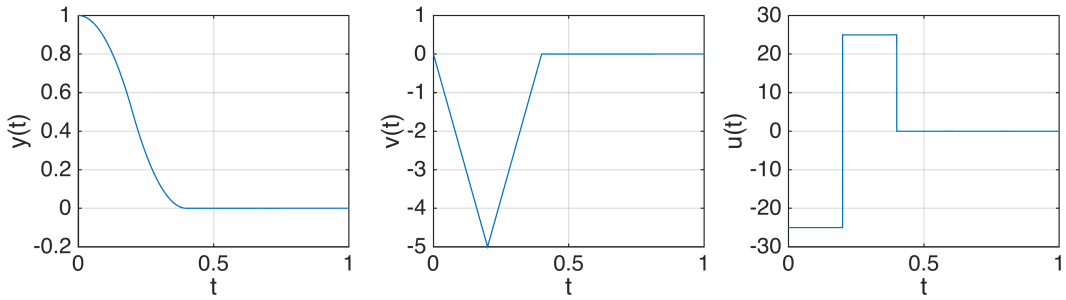


Fig. 9: As $R \to 0$, deadbeat control: the position and velocity both get very close to zero after two steps ($t = 0.4$), at the cost of very large control values (up to ± 25).

Can we always drive the state to zero in just two sampling periods? In principle yes, but in practice this typically requires unreasonably large actuation. Actuators have physical limits that are not captured by this linear, unconstrained LQR formulation; within this framework, the usual workaround is to increasingly penalize the control input until the resulting actuation respects the actual constraints. This point is revisited in later lectures.

! Interactive example: tune the double integrator yourself

Rather than take these tuning guidelines on trust, it is worth turning the knobs yourself. The demo below runs exactly the computation of this section: it iterates the Riccati recursion backwards from $P_h = Q_h$, applies $u_k = K_k x_k$ from $x_0 = [1 \;\; 0]^\top$, and draws the resulting $y(t)$, $v(t)$ and $u(t)$ – the continuous curves between sampling instants are exact, since the input is constant on each interval. The rectangle is the mass sliding on its track, with the red arrow showing the force u and the dashed green line marking the target $y = 0$.

With the values it starts from – $Q = I$, $R = 1$, $Q_h = 10I$, $h = 5$ – it reproduces the worked example above exactly: $K_0 = [-0.9179 \;\; -1.5030]$, an optimal cost $J_0(x_0) = 12.1214$ (which is just $x_0^\top P_0 x_0$, the $(1,1)$ entry of the P_0 printed above), and a mass that has only crawled to $y \approx 0.75$ by the end of the horizon.

From there, each of the three guidelines is one number away. Raise the terminal weight to $Q_h = 100I$ and the final position drops to 0.370; raise it to $1000I$ and it drops to 0.063. Raise the position weight q_1 relative to the velocity weight q_2 and the mass arrives sooner while the velocity error grows. Lower R and the controller buys speed with actuation. Two further things are worth trying, both of which anticipate the next section: push the horizon h up and watch K_0 stop changing – at $h = 60$ it has settled on $[-0.8412 \;\; -1.5460]$, the infinite-horizon gain – and push R towards zero to see how much actuation the controller will spend when nothing restrains it.

Increasing the horizon and the infinite-horizon LQR

Consider now the effect of the horizon length h, fixing $Q = Q_h = I$, $R = 1$ and $x_0 = [1 \;\; 0]^\top$.

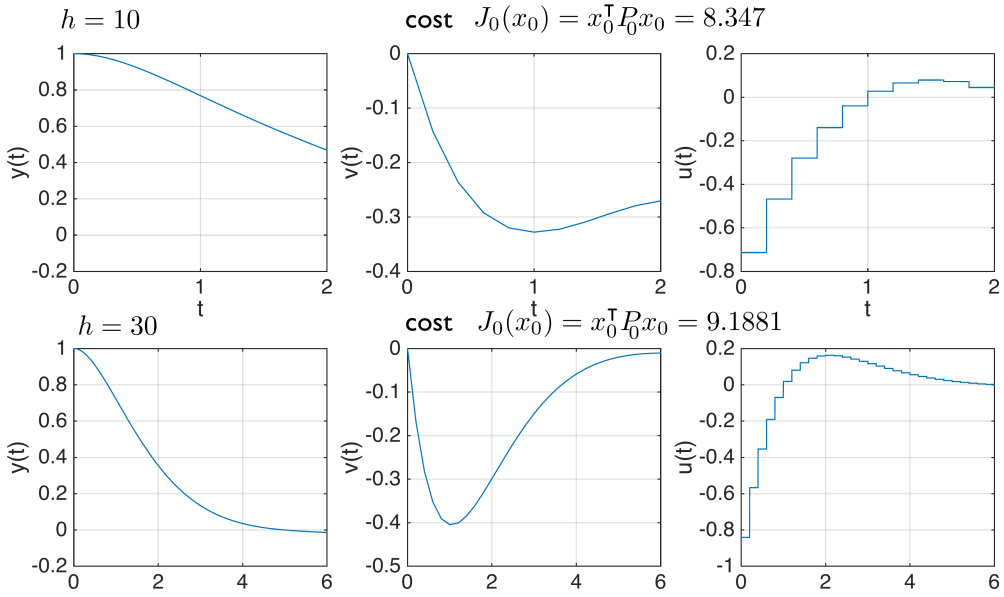


Fig. 10: Optimal trajectories for horizons $h = 10$ (top, $t \in [0, 2]$) and $h = 30$ (bottom, $t \in [0, 6]$). The optimal cost $J_0(x_0) = x_0^\top P_0 x_0$ grows from 8.347 ($h = 10$) to 9.1881 ($h = 30$).

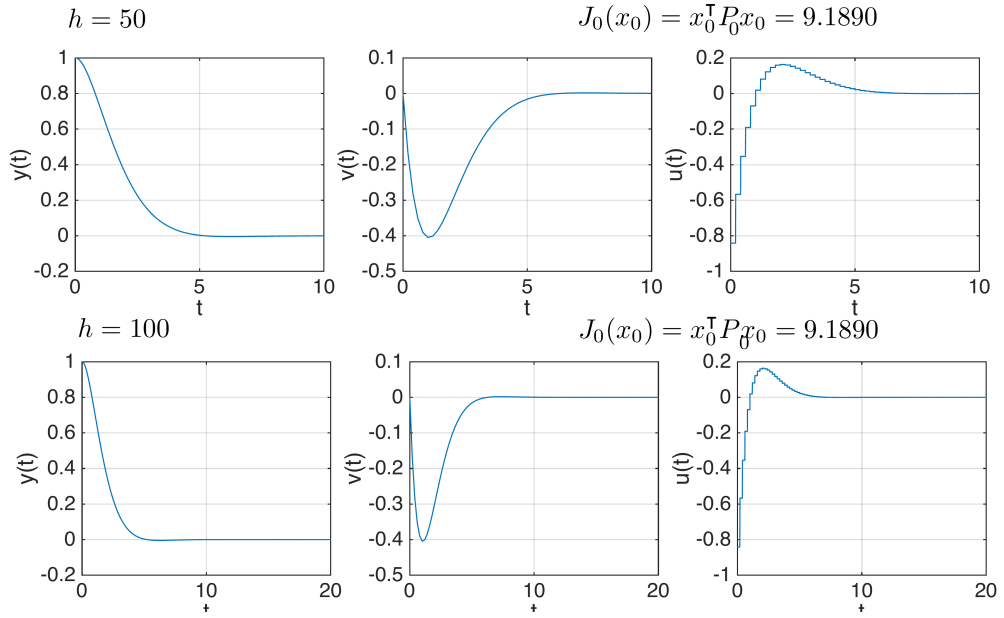


Fig. 11: Optimal trajectories for horizons $h = 50$ (top, $t \in [0, 10]$) and $h = 100$ (bottom, $t \in [0, 20]$). The optimal cost is 9.1890 in both cases: it has essentially converged.

The cost converges to a constant as the horizon increases. This makes sense: since the running cost is positive-definite, if the horizon is long enough the optimal input should drive the state to (approximately) zero well before the end of the horizon, simply to stop paying cost. That is,

$$\sum_{k=0}^{h_0} g_k(x_k, K_k u_k) + \underbrace{\sum_{k=h_0+1}^{h-1} g_k(x_k, K_k u_k) + g_h(x_h)}_{\approx\ 0 \text{ since } x_k \approx 0 \text{ for } k > h_0},$$

so lengthening the horizon beyond h_0 adds almost nothing to the cost. This reasoning holds for every initial condition, so if $x_0^\top P_0 x_0$ converges as $h \to \infty$ for every x_0, then P_0 itself converges, where P_0 is obtained from the (now time-invariant, since we take A_k, B_k, Q_k, R_k, S_k constant) recursion

$$P_k = A^\top P_{k+1} A + Q - (S + A^\top P_{k+1} B)(B^\top P_{k+1} B + R)^{-1}(S^\top + B^\top P_{k+1} A), \qquad k \in \{h-1, h-2, \dots, 0\}.$$

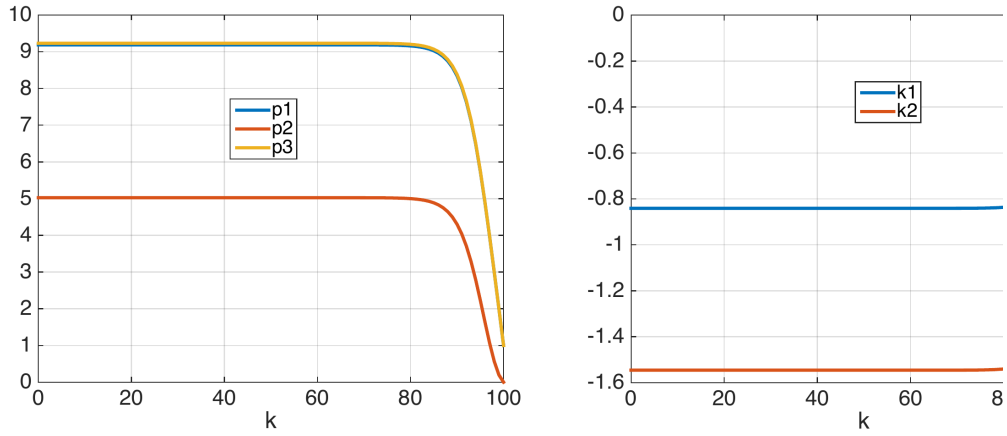


Fig. 12: For the double integrator with $Q = Q_h = I$, $R = 1$ and $h = 100$: the entries of $P_k = \begin{bmatrix} p_{1,k} & p_{2,k} \\ p_{2,k} & p_{3,k} \end{bmatrix}$ (left) and of $K_k = \begin{bmatrix} K_{1,k} & K_{2,k} \end{bmatrix}$ (right), plotted against k. Both are essentially constant except very close to the terminal stage $k = h$, where P_k must satisfy the boundary condition $P_h = Q_h$.

Denoting by $\bar{P}$ the limit of P_0 as the horizon grows, $\bar{P}$ must satisfy the *algebraic Riccati equation*

$$\bar{P} = A^\top \bar{P} A + Q - (S + A^\top \bar{P} B)(B^\top \bar{P} B + R)^{-1}(S^\top + B^\top \bar{P} A),$$

and correspondingly $K_k \to \bar{K}$ with

$$\bar{K} = -(B^\top \bar{P} B + R)^{-1}(S^\top + B^\top \bar{P} A).$$

This is precisely the infinite-horizon linear quadratic regulator.

! Interactive example: does the Riccati recursion always converge?

The figure above is one particular problem. The point worth internalizing is that nothing about it was special: under mild conditions – (A, B) controllable, R positive definite, and (A, N) observable where $Q = NN^\top$ – the recursion converges, and converges *quickly*, whatever A, B, Q and R you start from. The P_k reach their limit within a few tens of steps and then sit there; only the last handful of stages before $k = h$ still remember the boundary condition $P_h = Q_h$.
The demo below lets you check that for yourself. Enter any A, B, $Q = \mathrm{diag}(q_1, q_2)$, R and horizon h (the terminal cost is taken as $Q_h = Q$) and it iterates the same recursion backwards from $k = h$,

plotting the entries of P_k and of K_k against k exactly as in Fig. 12. Underneath it reports the limit $\bar{P}$, the limit gain $\bar{K}$, how many steps back from $k = h$ the recursion took to settle, and – this is the part that anticipates the proposition below – the spectral radius $\rho(A + B\bar{K})$ of the resulting closed loop, which comes out strictly less than one whenever the conditions hold.
Three things are worth trying. Start from the double integrator it opens with and change the weights over orders of magnitude: $Q = 10^4 I$, or $R = 10^3$, or $q_2 = 0$ – the limit $\bar{P}$ changes completely but it is always *reached*, and the closed loop is always stable. Then load the very unstable plant $A = \begin{bmatrix} 3 & 1 \\ 0 & 2 \end{bmatrix}$: convergence is, if anything, faster – about 16 steps rather than 61 – since an unstable plant is expensive to leave alone and the optimal gain is decided quickly. Finally load the uncontrollable preset, where $A = \mathrm{diag}(2, 0.5)$ and $B = [0 \;\; 1]^\top$ leave the unstable mode x_1 untouched by the input: now P_k simply runs away as the recursion goes back, there is no finite $\bar{P}$, and the infinite-horizon problem has no solution. That is the role the controllability assumption plays in the proposition that follows.

Proposition (special case of [Bertsekas, Sec. 4, Proposition 4.4.1]). The optimal policy for the stage decision problem with an infinite number of stages, dynamic model

$$x_{k+1} = Ax_k + Bu_k, \qquad (A, B) \text{ controllable},$$

and cost function

$$\sum_{k=0}^{\infty} x_k^\top Q x_k + 2x_k^\top S u_k + u_k^\top R u_k, \qquad \begin{bmatrix} Q & S \\ S^\top & R \end{bmatrix} > 0,$$

is given by

$$u_k = \bar{K} x_k, \qquad \bar{K} = -(B^\top \bar{P} B + R)^{-1}(S^\top + B^\top \bar{P} A),$$

where $\bar{P}$ is the unique positive definite solution of the algebraic Riccati equation above. Furthermore, the closed-loop system $x_{k+1} = (A + BK)x_k$ is exponentially stable.

The assumptions of this proposition can be relaxed considerably. As noted in [Bertsekas, Sec. 4.1], for simplicity take $S = 0$. If (A, B) is controllable, R is positive definite, $Q = NN^\top$ for some full rank $N \in \mathbb{R}^{n \times r}$ (so Q need not itself be positive definite when $r < n$), and (A, N) is observable, the theorem still holds. If we relax further – R positive definite, (A, B) merely *stabilizable*, N full rank, and (A, N) only *detectable* – the theorem still holds, except that $\bar{P}$ need not be positive definite. In fact, following [B. O. Anderson, J. B. Moore, *Linear Optimal Control*, Sec. 14.1], it suffices that $B^\top Q B + R$ be positive definite and (A, N) observable to guarantee closed-loop stability. In particular, one may take $R = 0$ and Q positive definite and still obtain a stable closed-loop system – a choice that would not be admissible for the continuous-time version of this problem.

Example: the inverted pendulum

As a more substantial example, consider the classic inverted pendulum on a cart: a cart of mass M moving on a track, actuated by a horizontal force u, carrying a pendulum of mass m, moment of inertia I and length ℓ, whose angle θ from the upright vertical is to be stabilized at zero together with the cart position x.

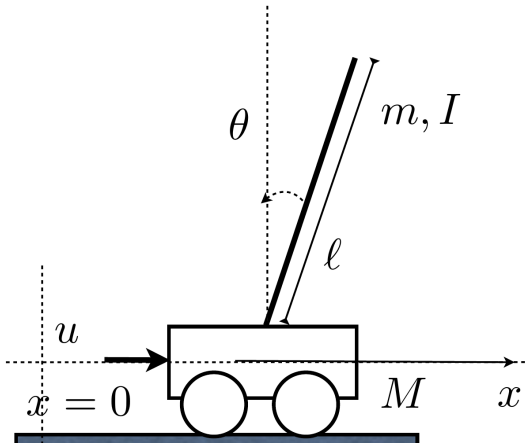


Fig. 13: The inverted-pendulum-on-a-cart system: a cart of mass M at position x, actuated by a force u, carrying a pendulum of mass m, inertia I and length ℓ at angle θ from the vertical.

The linearized equations of motion about the upright equilibrium are (see [1, p. 32])

$$(I + m\ell^2)\ddot{\theta} - mg\ell\theta = m\ell\ddot{x}, \qquad (M + m)\ddot{x} + b\dot{x} - m\ell\ddot{\theta} = u,$$

which can be written in state-space form, with state $[x \;\; \dot{x} \;\; \theta \;\; \dot{\theta}]^\top$, as

$$\frac{d}{dt}\begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & -\frac{(I+m\ell^2)b}{q} & \frac{m^2 g \ell^2}{q} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & -\frac{m\ell b}{q} & \frac{mg\ell(M+m)}{q} & 0 \end{bmatrix} \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{I+m\ell^2}{q} \\ 0 \\ \frac{m\ell}{q} \end{bmatrix} u(t), \qquad q = (I + m\ell^2)(M + m) - m^2\ell^2.$$

[1] *Feedback Control of Dynamic Systems*, Franklin, Powell, Emami-Naeini.

Discretizing this model and applying the (finite-horizon) LQR design with $Q = I$, $S = 0$, $R = 1$ and sampling period $\tau = 0.1$ gives the closed-loop response below, starting from a small initial displacement of the cart.

$$Q = I, S = 0, R = 1, \tau = 0.1$$

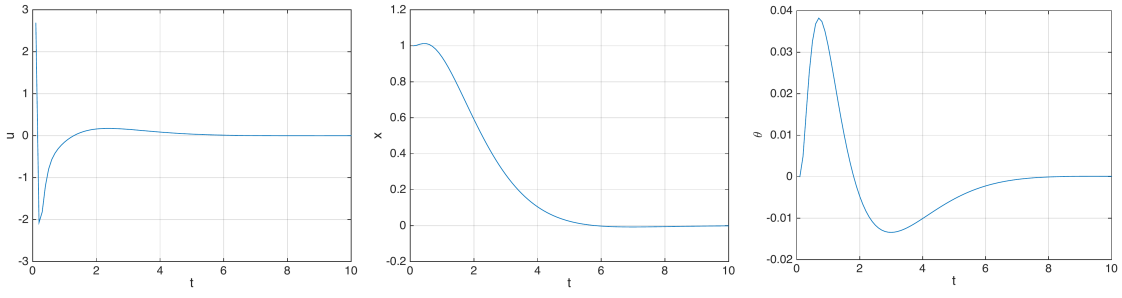


Fig. 14: Closed-loop response of the linearized inverted pendulum under LQR feedback, with $Q = I$, $S = 0$, $R = 1$, $\tau = 0.1$: control input $u(t)$, cart position $x(t)$ and pendulum angle $\theta(t)$. Both the cart position and the pendulum angle are driven to zero within a few seconds, with only a small transient swing of the pendulum.

As with the double integrator, the closed-loop behavior can be reshaped by tuning the cost weights: the figure below compares two further choices of weights, illustrating the same trade-off between response speed and actuation effort observed earlier.

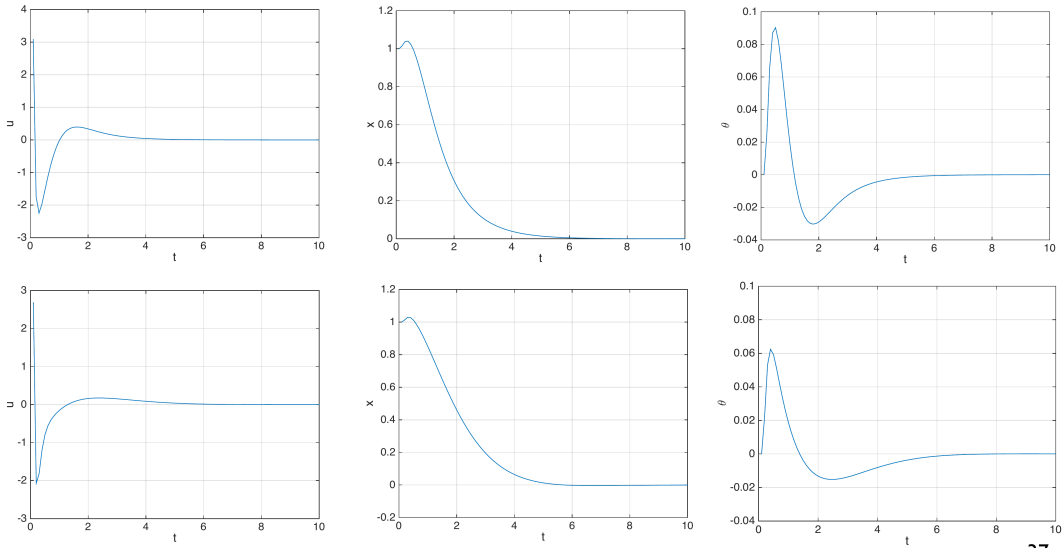


Fig. 15: Two further tuning choices for the inverted-pendulum LQR design, each showing $u(t)$, $x(t)$ and $\theta(t)$: as with the double integrator, more aggressive weights on the state trade off against faster settling and larger control effort.

! Interactive example: balance the inverted pendulum yourself

The figures above are fixed recordings of one design. The simulation below lets you fly the same closed loop yourself, with the gain K obtained by iterating the Riccati recursion to convergence – the infinite-horizon LQR of the previous section. It uses the same inverted-pendulum-on-a-cart system as Fig. 13, with a standard textbook parameter set (cart mass $M = 0.5$ kg, pendulum mass $m = 0.2$ kg, pendulum inertia $I = 0.006$ kg·m^2 about its own center of mass, pendulum length $\ell = 0.3$ m to that center of mass, cart friction $b = 0.1$ N · s/m, and $g = 9.8$ m/s^2). Choose a state weight $Q = qI$ and a control weight R and click "Apply" to design a gain K; then push the cart left or right and watch the controller react. As long as the pushes stop, the closed loop – being stable – always drives the cart and pendulum back to the upright equilibrium, whatever Q and R you chose; larger Q relative to R reacts faster but spends more control effort, and vice versa.

Extensions and applications

The linear quadratic regulator is one of the very few cases in which the value functions and the policy of the dynamic programming algorithm can be computed *exactly* on a continuous state and input space. That is unusual enough to be worth exploiting, and it is why the framework has so many extensions and applications; several of them are explored in the optional exercises. Here we look at two. The first extends the LQR from *stabilization*, where the goal is to drive the state to zero, to *reference tracking*, where it must follow a prescribed signal. The second is a related and rather less obvious use of the same machinery: generating smooth trajectories.

Reference tracking. Replace the cost of the stabilization problem by

$$\sum_{k=0}^{h-1} \|Mx_k - r_k\|^2 + u_k^\top R u_k + \beta_h \|Mx_h - r_h\|^2,$$

where r_k is the reference for the output Mx_k. The cost is no longer a pure quadratic form in x: it now has a linear term as well, so the cost-to-go takes the affine-quadratic form $J_k(x_k) = x_k^\top P_k x_k + 2b_k^\top x_k + c_k$, and running the same argument as before gives an optimal policy that is *affine* rather than linear,

$$u_k = K_k x_k + L_k,$$

with the feedback gains K_k given by exactly the Riccati recursion of the stabilization problem with $Q = M^\top M$, and a feedforward term $L_k = -(R + B^\top P_{k+1} B)^{-1} B^\top b_{k+1}$ driven by the vector b_k, which runs backwards according to $b_k = -M^\top r_k + (A + BK_k)^\top b_{k+1}$. The whole influence of the reference is carried by b_k, and hence by L_k: the feedback gains do not depend on r at all.

Trajectory generation. Now a less obvious application of the same result. Suppose we are given a sequence of waypoints that a vehicle must pass through, and we want a trajectory through them that is *smooth* – with derivatives existing up to some order. Take as the model a chain of integrators, so that the state collects the position and its first $n-1$ derivatives and the input u_k is the top derivative, and take as the reference r_k the piecewise-linear interpolant through the waypoints. Minimizing the tracking cost above then trades off staying close to that interpolant against using a large top derivative, and the result is a smooth curve passing near the waypoints. Trajectory design has become a tracking problem, solved by exactly the recursion we already have.

In the demo below each axis is a chain of three integrators, so the input is the jerk and the trajectory has continuous position, velocity and acceleration. Click on the canvas to add waypoints; the dashed grey line is the piecewise-linear interpolant and the blue curve is the trajectory the LQR produces. The single knob is R: lower it and the curve is pulled tightly onto the corners at the cost of large jerk, raise it and the curve relaxes into a wide, smooth sweep that cuts the corners.

Summary

Stage decision problems generalize the discrete optimization problems of Units 1-4 to arbitrary state and input spaces, and the dynamic programming algorithm carries over unchanged: it still computes an optimal policy by working backwards from a terminal cost, one stage at a time. In general, however, the cost-to-go J_k can be an arbitrarily complicated function of x_k, so applying the DP equation explicitly may be intractable. The linear quadratic regulator is the important exception to this rule: when the dynamic model is linear and the cost is quadratic, the cost-to-go remains a quadratic form $x_k^\top P_k x_k$ at every stage, and the DP recursion collapses into the Riccati recursion for P_k together with a linear state-feedback policy $u_k = K_k x_k$. As the horizon grows, P_k and K_k converge to the solution of an algebraic Riccati equation and to a fixed gain $\bar{K}$, yielding the infinite-horizon LQR and a stabilizing linear feedback law for any controllable (or merely stabilizable) linear plant – as illustrated here for the double integrator and the inverted pendulum, and as will be used throughout the rest of the course as the standard state-feedback design tool.